Exp. No 12

25/9/24

END-TO-END COMMUNICATION AT TRANSPORT LAYER

Q) Echo client-server using TCP/UDP sockets.

Aim: To implement an echo client-server program using TCP/UDP sockets.

PROCEDURE:

1. Import the socket module
2. Create a server socket and bind it to an IP address and port.
3. Start the server and wait for client messages.
4. The client sends a message to the server.
5. The server sends the same message back to the client.
6. Display the received message.

TCP/UDP Sockets:

TCP and UDP sockets are the endpoints used by applications to send and receive data across a network. They operate at the Transport layer (layer 4) of the OSI model, acting as the bridge between your software code and the physical network infrastructure.

PROGRAM: - server.py (TCP)

```
import socket
s = socket.socket()
s.bind(("127.0.0.1", 5000))
s.listen(1)
c, addr = s.accept()
msg = c.recv(1024).decode()
print("client", msg)
c.send(msg.encode())
c.close()
s.close()
```

PROGRAM - client.py (TCP)

```
import socket
s = socket.socket()
s.connect(("127.0.0.1", 8000))
msg = input("Enter message:")
s.send(msg.encode())
print("Echo:", s.recv(1024).decode())
s.close
```

input:

Enter message: Hello

output:
client: Hello
Echo : Hello

RESULT: Thus the echo client-server program was implemented using TCP sockets.

25/9/20

b) Chat Program using Socket Programming

Aim:

To implement a chat program using client-server Socket programming.

PROCEDURE:

1. Import the Socket module
2. Create a server Socket and bind it to a port
3. Start the server and accept a client connection
4. The client sends a message to the server
5. The server replies to the client.
6. Display the exchanged messages and close the connection.

SOCKET:

A socket is an abstract software endpoint that allows two processes - either on the same computer or across the globe via the internet to communicate with each other.

CODE - server.py (TCP)

```
import socket
s = socket.socket()
s.bind(("127.0.0.1", 5001))
s.listen()
c, addr = s.accept()
while True:
    msg = c.recv(1024).decode()
    if msg.lower() == "bye":
        break
    print("client", msg)
    reply = input("Server : ")
    c.send(reply.encode())
    if reply.lower() == "bye":
        break
c.close() s.close()
```

CODE - client.py (TCP)

```
s = socket.socket()
s.connect(("127.0.0.1", 5001))
while True:
    msg = input("client: ")
    s.send(msg.encode())
    if msg.lower() == "bye":
        break
    reply = s.recv(1024).decode()
    print("Server", reply)
    if reply.lower() == "bye":
        break
s.close()
```

Input:

Client: Hello

Server: Hi

Client: How are you?

Server: Fine

Client: bye

output:

Client: Hello

Server: Hi

Client: How are you?

Server: Fine

RESULT:

Thus a client - Server chat program was implemented Successfully using TCP sockets.